

Optimización de salida de rutas universitarias mediante predicción de demanda y aprendizaje por exploración/explotación, con apoyo de IA generativa

1. Problema

Dos rutas de transporte hacia la UPP salen de dos puntos distintos (central de autobuses de Pachuca y Puente de Tuzos en Mineral de la Reforma) y llegan al mismo punto en la universidad. En fin de cuatrimestre, los tiempos de salida se vuelven impredecibles porque las combis esperan a llenarse con 18 personas. Una de las rutas (Puente de Tuzos en Mineral de la Reforma) tiene llegada más lenta de alumnos, aproximadamente uno cada 7 minutos, lo que causa salidas muy tardías.

2. Objetivo

Diseñar un sistema que recomiende cuándo debe salir cada combi para reducir el tiempo de espera de los alumnos, manteniendo una ocupación razonable del transporte.

3. Datos necesarios

Los alumnos simulan:

Datos de demanda de fin de cuatrimestre:

- Fecha.
- Hora.
- Ruta.
- Número de alumnos esperando en la combi.
- Cantidad de alumnos que llegan por intervalo de tiempo.

Datos de despacho de fin de cuatrimestre de cada combi:

- Hora de salida de la combi.
- Número de pasajeros al salir.
- Si se hizo salida parcial o salida llena.
- Tiempo de espera acumulado.

Datos de ruta:

- Tiempo de recorrido.
- Paradas intermedias.
- Alumnos recogidos en paradas intermedias.
- Hora de llegada a la universidad.

4. Realizar un modelo con regresión lineal simple

Algunos ejemplos son:

Modelo 1: demanda por hora

$$\text{alumnos_en_15min} = a + b \times \text{hora}$$

Esto permite estimar cuántos alumnos llegarán próximamente.

Modelo 2: tiempo para llenar combi

$$\text{minutos_para_llenar} = a + b \times \text{personas_esperando_en_combi}$$

Esto permite estimar cuánto conviene esperar.

Modelo 3: tiempo de recorrido

$$\text{tiempo_recorrido} = a + b \times \text{hora_salida}$$

Esto permite estimar la llegada a la universidad.

5. Realizar UCB

Implementar UCB para simular las últimas tres semanas del cuatrimestre actual y ver que política conviene más en lugar de la política de “salir con combi llena”.

Estado del sistema:

En cada momento, el sistema podría observar (seleccionar un estado):

- Ruta.
- Hora.
- Personas esperando en combi.
- Tiempo transcurrido desde que llegó la combi.
- Tasa estimada de llegada.
- Predicción de demanda.
- Ocupación actual.

Acciones (Brazos):

El sistema puede elegir entre políticas como:

- Si hay 18 personas en combi, salir inmediatamente.
- Si hay 14 o más personas en combi y pasaron 15 minutos, salir.
- Si hay 12 o más personas en combi y pasaron 25 minutos, salir.
- Si hay 10 o más personas en combi y pasaron 35 minutos, salir.
- Si hay menos de 10 personas en combi, esperar máximo 45 minutos o hasta alcanzar un mínimo razonable.

Recompensa:

Una recompensa posible:

$$R = -w_1 \times \text{espera_total} - w_2 \times \text{alumnos_sin_transporte} - w_3 \times \text{asientos_vacíos}$$

Donde:

- w_1 : peso del tiempo de espera.
- w_2 : penalización por dejar alumnos.
- w_3 : penalización por baja ocupación.

Para usar UCB fácilmente, pueden normalizar la recompensa entre 0 y 1.

El sistema debe aprender cuál es el mejor balance según la hora, la demanda y las prioridades de la universidad en fin de cuatrimestre.

6. Entregables

El equipo debe entregar:

1. Dataset simulado de demanda y salidas de las dos rutas.
2. Modelo de regresión lineal simple.
3. Implementación de UCB para elegir la mejor política de salida de una combi.
4. Bitácora de IA.
5. Presentación en formato PDF.

7. Métricas de evaluación del proyecto

Algunas métricas recomendadas son:

Para alumnos:

- Tiempo promedio de espera.
- Tiempo máximo de espera.
- Alumnos transportados.
- Alumnos que se quedaron sin transporte.
- Hora promedio de llegada a la universidad.

Para operación:

- Ocupación promedio del camión.
- Porcentaje de asientos vacíos.
- Número de salidas necesarias.
- Costo operativo aproximado.
- Balance entre rutas.

Para el algoritmo:

- Recompensa acumulada.
- Comparación UCB vs política actual.

8. Resultados esperados

Los alumnos deben responder una o varias preguntas como:

- ¿Conviene seguir esperando a que la combi se llene con 18?
- ¿Cuál es el número mínimo razonable de pasajeros para que la combi salga?
- ¿Cuál es el tiempo máximo de espera aceptable?
- ¿En qué horas conviene ser más flexible?

- ¿Cuánto tiempo de espera se puede reducir si se sale con 12 o 14 personas?

9. Uso de IA generativa

Los estudiantes pueden usar IA generativa como apoyo, pero deberán entregar como evidencia una bitácora de IA.

La bitácora de IA es una tabla donde por cada registro, los estudiantes documentan cómo usaron la inteligencia artificial durante el proyecto. No debe ser solo “copiar y pegar lo que dijo la IA”, sino mostrar que preguntaron, revisaron, corrigieron y aprendieron.

Información de un registro de una bitácora de IA

1. Fecha:
2. Herramienta:
3. Tema:
4. ¿Qué le pregunté?
5. ¿Qué me respondió?
6. ¿Qué verifiqué?
7. ¿Qué corregí?
8. ¿Qué aprendí?

Ejemplo Bitácora de IA

Instrucciones de uso

1. Cada interacción significativa con una herramienta de IA generativa debe quedar registrada como un renglón independiente.
2. No se trata de copiar y pegar la respuesta de la IA. Se evalúa que el alumno pregunte, analice, verifique, corrija y aprenda.
3. Se espera un mínimo de 8 registros a lo largo del proyecto (al menos 2 por cada entregable principal: dataset, regresión, UCB y presentación).
4. La bitácora se entrega junto con el proyecto final, en formato digital (PDF o Word).
5. Los registros deben ser cronológicos (del más antiguo al más reciente).

Ejemplo de bitácora de IA

Fecha	Herramienta	Tema	¿Qué le pregunté?	¿Qué me respondió? (resumen)	¿Qué verifiqué?	¿Qué corregí?	¿Qué aprendí?
14/08/2026	ChatGPT (GPT-4o)	Generación del dataset simulado de demanda	Necesito simular la llegada de alumnos a una parada de combi universitaria durante fin de cuatrimestre. La ruta Puente de Tuzos tiene una llegada aproximada de 1 alumno cada 7 minutos. ¿Cómo generó en Python un dataset con columnas: fecha, hora, ruta, alumnos_esperando, llegadas_por_intervalo, para 21 días?	Propuso usar <code>numpy.random.poisson</code> con $\lambda \approx 2.14$ (60/7) para modelar llegadas por intervalo de 15 min. Dio un código que generaba 21 x 96 intervalos y acumulaba alumnos esperando.	Revisé que la distribución Poisson fuera adecuada para llegadas independientes. Comparé la media generada (≈ 2.1 alumnos/15 min) con el dato del problema (1 cada 7 min ≈ 2.14). Consulté el libro de Introduction to Probability de Blitzstein para confirmar que Poisson modela llegadas en intervalos fijos.	El código original no distinguía entre hora pico (7:00-9:00 y 13:00-15:00) y hora valle. Ajusté λ por franja horaria: $\lambda = 3.5$ en pico, $\lambda = 1.2$ en valle. También agregué la columna ruta que faltaba.	Que una distribución Poisson con λ variable por franja horaria es más realista que una λ fija. También aprendí a usar <code>numpy.random.default_rng()</code> en lugar de <code>np.random.seed()</code> para reproducibilidad moderna.
14/08/2026	Copilot	Modelo de regresión: minutos para llenar combi	Tengo un DataFrame con columnas "personas_esperando_en_combi" y "minutos_para_llenar". ¿Cómo ajusto una regresión lineal simple con scikit-learn y obtengo los coeficientes a y b?"	Generó código con "LinearRegression().fit(X, y)", imprimió "coef_" e "intercept_", y graficó la recta con matplotlib.	Ejecuté el código y obtuve $b \approx -1.8$, lo que implicaba que a más personas esperando, menos minutos para llenar. Eso tiene sentido lógico. Verifiqué el $R^2 = 0.74$. Revisé en la documentación de scikit-learn que "coef_" devuelve un array aunque sea una sola variable.	El código de Copilot usaba "X = df[['personas']]" (Serie 1D) y scikit-learn requiere 2D. Lo cambié a "X = df[['personas_esperando_en_combi']]" (matriz 2D). Además, el gráfico no tenía etiquetas en los ejes; las agregué.	Que scikit-learn exige arrays 2D para las features, incluso en regresión simple. También que un R^2 de 0.74 es aceptable para datos simulados con ruido, pero debo mencionar la limitación en la presentación

10. Rúbrica de evaluación

Criterio	Puntos
Modelo de regresión lineal simple	10
Bitácora de IA	15
DataSet Simulado	15
Presentación	20
Implementación con UCB	10